Clean_exp — The Whole Story

A plain-language walkthrough of what the repo contains, what goes in, what happens at each stage, which prompt runs where, and a real worked example showing the retry loop end to end.

---

1. What this repo actually does

You give it a Software Requirements Specification (SRS) — a document describing what a piece of software is supposed to do — and the real source code of that software. The pipeline asks several open-weight LLMs to *read only the SRS* (never the code) and write a full test suite for it, in one of 30 possible test-case notations (Gherkin, Decision Tables, xUnit, Use-Case scenarios, state machines, and so on). Each generated suite is pushed through three automated quality gates. Only after a suite is completely frozen does the pipeline finally open the real code and run the tests against it, to see whether a suite that *looked* good actually catches anything real.

The three case-study projects are **doorstop** (a requirements-management tool), **ventilator**, and **virtual_clinic**. 10 different LLMs were run across all three, producing 496 test suites and 4,058 individual test cases in total.

2. What's in the repo

Top-level layout, in the order you'd actually look at things:

Folder / file	What it is
config.yaml	The control panel: which LLM providers/models are used for what role (generator / judge / extractor / executor), gate thresholds, repair budget, paths. No secrets live here — those come from a separate `.env` file.
tcgen/	The actual pipeline code. One Python module per stage (`ingest.py`, `extract.py`, `represent.py`, `suite.py`, `execute.py`, `report.py`), plus `gates/` which holds the scoring logic for all three gates, and `llm.py`, one client used for every provider.
prompts/	Every prompt the pipeline sends to an LLM, as plain .txt files with `{{PLACEHOLDERS}}`. Nothing is hard-coded inside the Python — if you want to see or change exactly what a model was asked, this folder is the source of truth.
schemas/	One JSON Schema file describing what a valid test case object must look like (id, title, requirement links, kind, steps, expected result, etc). Used to mechanically check generated output.
runs/ANALYSIS/	The aggregated results of the full benchmark: `suites.csv` (one row per generated suite, 496 rows), `REPORT.md` (the human-readable summary), and per-model/per-representation breakdowns. The raw per-suite working files (one folder per model per project, 150MB+) are *not* in this repo — only this aggregated layer is.
tools/	Shell scripts for talking to the SERC GPU server that actually ran the open-weight models (SSH tunnel keep-alive, etc).
docs/PIPELINE.md	The detailed internal design notes: what each stage does, why, its pros/cons, and what to look at when sanity-checking it. This document is a simplified companion to that one.
paper/	The written-up research paper (LaTeX + PDF) built from the `runs/ANALYSIS/` numbers, plus the slide decks used to present the findings.

3. What goes in

Every case study lives under `inputs/<project>/`, outside this repo (kept separate so the benchmark subjects aren't bundled with the pipeline code):

Input	What it holds
`srs/`	The requirements document(s) — PDF, DOCX, Markdown or plain text. Several files are fine; they all get merged into one text blob in stage S0.
`code/`	The real implementation. This is **never touched until stage S6** — every earlier stage is only allowed to read the SRS.
`project.yaml`	Optional: language, framework, how to set up and run the test command, for non-Python projects.

The one rule that matters most in the whole design: stages S0 through S5 are only allowed to read the SRS text. The real source code is opened for the very first time in stage S6, after every suite from every model has already been frozen and fingerprinted. This stops a model from "peeking" at the real function signatures and writing suspiciously accurate tests — it has to guess the interface purely from the English requirements, exactly like a real test engineer working from a spec before the code exists.

4. The pipeline, stage by stage

Each stage writes its output to a file under `runs/<project>/`, so any single stage can be re-run on its own without redoing earlier ones.

Stage	Code	Prompt file used	What happens
S0 Ingest	`ingest.py`	—	Every SRS file (PDF/DOCX/MD/TXT) is converted to one clean text file. No LLM call.
S1 Extract	`extract.py`	extract.txt	A fixed "extractor" model reads the SRS in ~7,000-character chunks and pulls out atomic, numbered requirements (id, section, text, kind, whether it's testable). Requirements are then grouped into "units" — roughly one SRS section's worth, capped at 8 requirements each.
S2 Gate 1 (RSS)	`represent.py`	select_reps.txt	For each unit, the *generator model being tested* picks the 2 best-suited test-case notations out of a 30-item catalog (Gherkin, Decision Tables, xUnit, Use-Cases, state machines, etc). An automated scoring heuristic (not an LLM) checks the pick against the requirement's subject matter, abstraction level and simplicity. A pick scoring below 0.60 is rejected and re-picked, up to 3 times.
S3 Generate	`suite.py`	generate.txt	The generator model writes the actual test suite in the chosen notation, plus the same cases again as a structured JSON list (so every notation can be checked and compared the same way downstream).
S4 Gate 2 (SFV)	`gates/sfv.py`	— (no LLM; pure code check)	Checks the JSON against the schema (valid fields, unique ids, kind is one of positive/negative/boundary) and checks the notation's own syntax (a real Gherkin parser for Gherkin, Python's own parser for xUnit code, structural checks for the rest). Needs a score ≥ 0.60 to pass.
S5 Gate 3 (FSA)	`gates/fsa.py`, groundedness.py	fsa_judge.txt	A separate "judge" model (a different model family from every generator, so no model grades its own homework) scores: clause coverage, negative/error-path coverage, how concrete the expected results are, boundary coverage, and — per concrete value asserted — whether that value is actually backed by the SRS text or invented. Needs FSA ≥ 0.50 *and* every requirement in the unit covered by at least one test.
repair loop	`suite.py`	feedback appended to generate.txt	If Gate 2 or Gate 3 fails, the specific problems found (schema errors, uncovered requirements, invented values, weak sub-scores) are fed straight back into the generation prompt and the model tries again — shared budget of up to 3 retries per suite across both gates. If nothing passes in time, the best attempt is kept and flagged as a fallback.
S6 Execute	`execute.py`	to_executable.txt, triage.txt	Only now is the real code opened. A fixed "executor" model translates the frozen, already-graded test cases into one runnable pytest file per batch, using the real API it can now see. Tests run for real; every failure is triaged by the LLM as a genuine code/spec mismatch, a bug in the generated test itself, or an ambiguous requirement.
S6A Audit (optional)	`audit.py`	audit.txt	A cheaper, execution-free alternative to S6: the judge just *reads* the code and guesses whether each test would pass, without actually running anything. Used for ventilator and virtual_clinic, which don't have a full pytest execution setup.

Stage	Code	Prompt file used	What happens
S7 Report	`report.py`	—	Rolls everything up per model and per representation: gate pass rates, FSA sub-scores, repairs used, execution pass rate, coverage, and how well the gate scores actually correlate with real execution results.

5. The three gates, in one line each

Gate	Pass threshold	What it's really asking
Gate 1 — RSS Representation Suitability	≥ 0.60	Is the chosen test notation actually the right tool for this requirement? (e.g. don't use a plain xUnit list for something that's fundamentally a state machine.)
Gate 2 — SFV Syntactic Form Validity	≥ 0.60	Is the output even syntactically valid — correct JSON schema, and valid syntax in the chosen notation? Pure code check, no LLM, free to run.
Gate 3 — FSA Functional Semantic Adequacy	≥ 0.50 + full coverage	Does the suite actually test the right things, with real checkable outcomes, including edge cases and negative paths — and are the concrete values it asserts actually grounded in the SRS, not invented?

Gates 1 and 3 use weighted formulas (different weights depending on whether the system looks "standard", safety-critical, data-pipeline-shaped, etc.) — the exact numbers are in `tcgen/gates/config.py`. The one weight worth knowing up front: for an ordinary ("standard") system, the groundedness sub-score's weight in the final FSA formula is **0.0** — meaning a suite can still pass Gate 3 with a perfect score even if every single concrete value it asserts turns out to be invented. The worked example below hits exactly this case.

6. A real worked example, start to finish

This is one real suite from the actual benchmark, not a hypothetical — **project: doorstop, model: Qwen2.5-Coder-32B, unit: U01_2, representation: "Use-Cases & User Stories"**. All numbers below are taken directly from that suite's saved run files.

Step 1 — S0/S1: the setup

Doorstop's SRS is ingested and chunked. Unit U01_2 ends up with 8 testable requirements about managing linkable, numbered text sections (creating sections, assigning permanent IDs, formatting, importing/exporting content, and so on).

Step 2 — S2: Gate 1 picks the notation

Qwen2.5-Coder-32B is asked (via `select_reps.txt`) to pick the 2 best representations for this unit. It picks "Use-Cases & User Stories" — a natural fit, since the requirements describe a human actor (a Requirements Engineer) going through multi-step scenarios. The RSS heuristic scores this pick above 0.60, so it passes Gate 1 immediately.

Step 3 — S3→S5: four attempts to get through Gates 2 and 3

This is the retry loop in action. Each row below is one full attempt — a fresh call to `generate.txt` (with the previous failure folded into `{{FEEDBACK}}` from attempt 2 onward), followed by a Gate 2 check and, once that passes, a Gate 3 judge call:

Attempt	Gate 2 (SFV)	Gate 3 (FSA)	What happened
0 (first try)	0.667 FAIL	— (never reached)	8 of 15 cases used "alternative" or "exception" as the test *kind* field — the schema only allows positive / negative / boundary. Gate 2 fails outright.
1 (repair)	0.833 FAIL	— (never reached)	The specific failing case numbers were listed back to the model. It fixed some, but 8 cases *still* used the invalid "exception" kind. Gate 2 still fails.
2 (repair)	1.000 PASS	0.0 FAIL	All 15 cases finally valid — Gate 2 passes. But the judge's response for Gate 3 came back truncated and couldn't be parsed as JSON, so the pipeline records that as an automatic FSA = 0.0 and fails the suite anyway, even though nothing was actually wrong with the test content yet.
3 (repair, final)	1.000 PASS	0.88 PASS	Same suite, re-judged cleanly this time: clause coverage 1.0, negative-path coverage 1.0, boundary coverage 1.0, oracle assertiveness only 0.2 (expected results are vague), and — of 7 concrete values the suite asserts (file names like "external_asset.docx", export formats "PDF"/"SVG", etc.) — **all 7 were judged not grounded in the SRS text, i.e. invented.** Because this project's system profile gives groundedness a weight of 0.0 in the FSA formula, none of that costs any points. FSA = 0.88, well above the 0.50 threshold. Suite is frozen.

That's the full repair budget used up (3 repairs is the maximum allowed per suite) — and it ends with a strong-looking pass: **FSA 0.88**, full requirement coverage, both gates green.

Step 4 — S6: the code is finally opened

Now, and only now, the real doorstop source code is copied into a throwaway workspace and the fixed executor model turns the 15 frozen test cases into one real pytest file. The results:

Result	Verdict	Why
1 / 15	Passed	The test's expectations matched real doorstop behaviour.
6 / 15	Failed — triaged "test_bug"	e.g. one test tried to construct a `Document` directly in a temp folder with no `.doorstop.yml` config file present, which doorstop requires — so the test crashed before it ever got to check the actual requirement.
8 / 15	Skipped — "NOT_IMPLEMENTED"	The executor model looked at the real code and couldn't find a public API for the feature being tested (e.g. "Formatting API not exposed in public API") and honestly flagged it rather than inventing a call.

The takeaway from this one suite: it sailed through every automated gate — Gate 1 suitability, Gate 2 syntax, Gate 3 adequacy at 0.88 — while every single one of its concrete assertions was ungrounded invention, and when actually run against the real implementation, only 1 of its 15 tests (about 7%) passed. This exact gap between "the gates say it's good" and "it actually works" is the central finding the paper (in `paper/`) is built around — measured here across all 496 suites, not just this one.

7. What's deliberately not in this repo

Left out	Why
Raw per-suite run folders (runs/doorstop/, etc.)	150MB+ of intermediate JSON per model per project — every attempt, every judge call. Kept out to keep the repo light; only the aggregated `runs/ANALYSIS/` layer (suites.csv, REPORT.md) is included.
LLM response cache (.cache/)	Regenerable — re-running the pipeline rebuilds it automatically since every call is cached by prompt hash.
Older pitch decks (presentations/)	Earlier, illustrative-numbers decks built for a different, non-technical audience before this real benchmark existed.
Secrets (.env)	API keys and GPU server credentials. Never committed — always read from an untracked `.env` file.

Generated as a companion explainer alongside `docs/PIPELINE.md` and the paper in `paper/`. All figures above are read directly from the repo's own source files and run outputs, not summarized from memory.